

Final Project Documentation

Project Title: OrganConnect (Organ Donation and Transplant Network Management System)

Domain: Web Development & Database Management Systems (DBMS)

Team Size: 5

Technologies Used: React (TypeScript), Tailwind CSS, Node.js (Express.js), MySQL

1. Abstract

Organ donation and transplantation is a highly time-sensitive process that saves lives. However, managing the data of donors, patients, doctors, and hospital organizations is often slow and disconnected. **OrganConnect** is a centralized full-stack web application designed to bridge this gap. This project provides role-based dashboards for Hospitals (Organizations), Doctors, Patients, and Donors. It maintains an updated inventory of organs, tracks patient medical histories, enables hospitals to match available organs with needy patients, and schedules procedures for doctors. By utilizing a robust MySQL database to enforce strict relationships and constraints, the system ensures data reliability, transparency, and speed in life-saving scenarios.

2. Introduction

In the medical field, the gap between organ availability and patient need requires careful management. A slight delay or mismanagement of data can cost lives. Currently, many hospitals manage their own separate databases, making it difficult to find matches efficiently. OrganConnect is developed to serve as a hub where donors can register, patients can submit their requirements, and hospitals can manage their doctors and transplants efficiently under a single unified platform.

3. Problem Statement

The existing systems for organ transplant management face the following issues:

- * **Fragmented Data:** Hospitals, patients, and donors are disconnected spread across different hospital-specific systems.
- * **Manual Errors:** Manual handling of complex associations (e.g., tying a single organ to exactly one patient and one doctor) leads to data redundancy and errors.
- * **Lack of Transparency:** Patients lack visibility into their requests, and organizations struggle to manage doctor schedules and organ inventory dynamically.

4. Objectives of the Project

- To design a secure, role-based authentication system for four specific user types: Donors, Patients, Doctors, and Organizations.
- To create a centralized database ensuring an organ can only be pledged once and transplanted once.
- To automate the workflow of matching a patient with an organ and automatically scheduling a consultation/surgery with a doctor.
- To provide interactive, modern UI dashboards for users to monitor their individual statuses.

5. Scope of the Project

The scope of OrganConnect is limited to the **informational and administrative workflow** of organ transplantation. * **In Scope:** User registrations, medical history logging, organ inventory management, doctor-to-hospital mapping, and transplant record creation (scheduling and billing). * **Out of Scope:** Real-time GPS tracking of physical organs during transport, integration with actual physical banking systems, and automated AI-based medical matching (matching is currently done administratively by the hospital).

6. Literature Review

Traditionally, hospital management systems (HMS) focus broadly on patient admission and billing. Specific organ tracking systems exist but are normally restricted locally to government agencies. Modern web technologies allow us to build decentralized nodes where multiple organizations can log into a shared “Hub.” By utilizing relational databases like MySQL, we ensure that ACID properties (which ensures data is safe and consistent even if the system crashes) prevent crucial errors, such as assigning an organ to two different people natively at the database level using **UNIQUE** constraints.

7. System Overview

OrganConnect is a web-based application. Users visit the website and choose their role to sign up or log in. Once authenticated: * **Donors** can pledge organs. * **Patients** submit medical details and request organs. * **Organizations** look at the global pool of organs and patients, assign their doctors, and create ‘Transplant’ events. * **Doctors** log in to see their assigned patients and mark surgeries as completed.

8. System Architecture

The project utilizes a **Client-Server Architecture** (3-Tier): 1. **Presentation Tier (Frontend):** Built with React.js and TypeScript. It uses Tailwind CSS for styling and runs via Vite for high performance. 2. **Logic Tier (Backend API):** Built with Node.js and Express.js. It acts as the bridge, receiving HTTP

requests from the frontend, processing business logic, and querying the database.

3. **Data Tier (Database):** Hosted on MySQL. This layer stores all user profiles, relationships, constraints, and audit logs.

9. End-to-End User Flow

This represents the step-by-step usage of the system by different roles to complete a successful donation cycle: 1. **Donor registers** and logs in → Adds an available organ pledge to the system. 2. **Patient registers** and logs in → Submits their medical requirement and history. 3. **Admin logs in** (representing the Hospital) → Views the inventory, and matches the available organ to the needy patient. 4. **Doctor logs in** (assigned by the Admin) → Views their schedule, performs the surgery, and confirms procedure completion. 5. **System updates** → The database automatically updates the organ's status to "Transplanted" and generates the final bill.

10. Data Flow / Pipeline Explanation

1. **User Registration:** Data flows from React Form → Express API → MySQL **Users** table (and the respective role table simultaneously).
2. **Organ Registration:** Donor updates profile → Submits Organ → Database marks organ as **Available**.
3. **Requirement Submission:** Patient updates Medical History → Hospital admin is notified.
4. **The Match (Core Flow):** The Hospital Admin views an **Available** organ and a Patient. The Admin creates a Transplant record.
5. **Database Trigger:** The system automatically changes the Organ status to **Reserved** and links the Patient, Doctor, and Organ in the **Transplant** and **Attends** tables.
6. **Execution:** The Doctor completes the surgery → System updates Organ status to **Transplanted** and Transplant status to **Completed**.

11. Modules Description

- **Authentication Module:** Handles secure login/signup using bcrypt for password hashing. Routes users to different dashboards based on their role.
- **Donor Module:** A simple interface allowing benevolent users to register their physical details and pledge organs into the system.
- **Patient Module:** Allows patients to maintain their medical history and contact information securely.
- **Organization Admin Module:** The core administrative hub. Admins can register new doctors under their hospital's ID, view holistic organ inventories, and execute the matching process.
- **Doctor Module:** A localized dashboard for doctors to view their schedule, patient medical histories, and finalize medical billing for completed

surgeries.

12. Technologies Used

- **Frontend:** **React** (Component-based UI), **TypeScript** (Prevents runtime bugs through strict type checking), **Tailwind CSS** (Fast, modern CSS styling).
- **Backend:** **Node.js** & **Express.js** (Lightweight, fast execution, JavaScript on the server).
- **Database:** **MySQL** (Chosen because the data is highly relational; an organ belongs to a donor, a transplant relies on a doctor, patient, and organ).

13. Database Design

The MySQL schema uses strong referential integrity: * **Users:** Core auth table. * **Role tables (Donor, Patient, Doctor, Organization):** Inherit from **Users** via **user_id** Foreign Keys. * **Organ:** Tracks the item. Uses **ENUM('available', 'reserved', 'transplanted')**. * **Transplant:** The central junction table. Uses **UNIQUE(organ_id)** to ensure physical impossibility of double-booking an organ. * **Medical_History** & **Attends:** Used for tracking patient records and doctor appointments.

14. Algorithms / Logic Used

The system relies primarily on **CRUD operations** (Create, Read, Update, Delete) and **SQL Joins** (combining data from multiple tables) rather than complex sorting algorithms. * **State Machine Logic:** The status of an organ explicitly follows a strict path: **Available** -> **Reserved** -> **Transplanted**. The backend explicitly prevents jumping states (e.g., you cannot transplant an available organ without reserving it for a procedure first). * **Authentication Logic:** Passwords are mathematically hashed using **bcrypt** and compared securely. Session IDs are generated using UUID strings.

15. Implementation Details

- **REST API:** The backend is divided into standard REST routes to seamlessly transfer JSON data to the frontend.
- **Connection Pooling:** The backend utilizes the **mysql2** pool to manage multiple simultaneous database queries efficiently without crashing the server.
- **Component Reusability:** The React frontend heavily utilizes reusable UI components (like standard buttons and input fields) to maintain design consistency and reduce code length.

16. API Endpoints Overview

Here are practical examples of the REST framework controlling the backend data:

- **POST /api/login:** Authenticates users based on their email and password, returning a secure session.
- **GET /api/organs/available:** Retrieves a JSON list of all organs currently marked as 'available' in the database.
- **POST /api/transplant:** Transmits the assignment details from the Organization to create a new transplant scheduling link.

17. Installation & Setup Guide

To run OrganConnect locally, follow these steps:

1. **Clone the repository** to your local machine.
2. **Install dependencies:**
Open your terminal in both the **frontend** and **backend** folders and run:
`npm install`
3. **Setup MySQL database:**
 - Create a local database named **Hospital**.
 - Import the provided SQL schema script to generate the required tables.
4. **Configure .env file:**
Create a **.env** file in the root backend directory to connect your database securely:

`DB_HOST=127.0.0.1`
`DB_USER=root`
`DB_PASSWORD=yourpassword`
`DB_NAME=Hospital`
5. **Run backend:**
Navigate to the backend directory and run:
`npm start`
6. **Run frontend:**
Open a separate terminal, navigate to the frontend directory, and run:
`npm run dev`

18. Results and Outputs

- Successfully built a responsive, highly accessible web interface.
- The database accurately denies invalid inputs (e.g., trying to use an organ twice throws an SQL Error, which the backend catches and turns into a user-friendly frontend message).

- Role-based routing successfully prevents a Patient from accessing the Organization dashboard.

19. Testing

- **Unit Testing:** Basic logic checking for functions and ensuring React components render properly.
- **Integration Testing:** Sending dummy requests using tools like Postman/ThunderClient to ensure the Express APIs write correctly to the MySQL DB.
- **UI/UX Testing:** Manual testing across different screen sizes to ensure Tailwind CSS responsiveness on mobile and desktop.

20. Advantages and Limitations

Advantages: * Completely digitalizes paper trails. * Enforces strict real-world rules at the database level (data cannot be manipulated to exist in two places).
* User-friendly interfaces reduce training time for administrative staff.

Limitations: * Currently relies on Organization Admins manually pairing organs with patients. * Relies on users correctly self-reporting medical history.

21. Future Scope

- **AI Matching:** Implementing a machine learning algorithm to automatically suggest the best Patient-Organ match based on blood type, age, and medical history.
- **Email/SMS Automation:** Integrating NodeMailer or Twilio so doctors and patients receive immediate text alerts when an organ match is made.
- **IoT Logistics:** Adding a module for delivery drivers to track the physical temperature and location of the organ block via GPS and IoT sensors.

22. Conclusion

The OrganConnect project successfully demonstrates how web technologies and relational databases can be combined to solve real-world logistical problems in the medical sector. By using React, Node.js, and MySQL, the system enforces secure, strict data handling, ensuring that the process of donating and receiving organs is transparent, trackable, and efficient.

23. References

- Express.js Official Documentation
- React and TypeScript Documentation
- MySQL Database Systems Guidelines
- MDN Web Docs (JavaScript specifications)